Abstraction

Abstraction is the foundational technique to manage complex systems: It simplifies complex systems into multiple hierarchical layers, allowing designers and programmers to focus on higher-level tasks while hiding the internal details of lower levels.

THE CHALLENGE

1900s

As computer systems grew in scale and capability, they became more complex. Computer architects and programmers had to find ways to work more efficiently.

Designing computers or say, writing complex programs would take dramatically longer if Software Designers / Programmers had to tinker around with barebones electrons.

THE Solution

Layer above Layer above Layer above... you get the idea

Abstraction solves this by dividing architecture into well-defined interface boundaries. Lower-level implementation details are concealed beneath intuitive higher-level abstractions.

Software engineers write code without needing silicon-level knowledge, while chip architects optimize hardware without altering application logic.

Abstraction ex.

Say you want to store the value 5 in a variable called x: You simply write `int x = 5;`
In reality, abstraction cloaks the memory mapping and physical transistor states required to fulfill that single command.

LEVEL 1 • HIGH-LEVEL CODE

Simple Programmer Syntax

```
int x = 5;
```

Written directly in high-level language ('C'). You simply declare and assign the variable without needing to worry about underlying hardware instructions or say **Assembly**.

Abstraction takes away the pain of thinking about **ARM** v/s **x86** v/s **Apple M** etc. Architectures.

Hides memory & CPU registers

LEVEL 2 • ARCHITECTURE & OS

Memory Location

```
MEM[0×7FFE] = 0×05;
```

In reality, the value 5 is stored in a memory location. *The programmer does not need to worry about the exact details / memory address inside RAM.*

This is where **abstraction** comes in, hiding the underlying hardware architecture stuffs, which are pretty complex.

Hides bus & physical addressing

LEVEL 3 • PHYSICAL HARDWARE

Voltages & Transistors

```
00000101 (High/Low V)
```

At the hardware level, 5 is represented using high and low **transistor voltages**, which **physically** store the binary value 00000101.

Abstraction allows you to simply use `int x = 5;` in your code, without having to know how to manage **silicon gates** / voltage states / **transistors** all by yourself.

Hides physical transistor physics